

- **Continuous improvement:** Code security tools often provide feedback and recommendations for improving code quality and security practices. By incorporating these suggestions into their development process, programmers can continually improve the security posture of their applications over time.

Pros and cons of open source code security tools

The pros are:

Transparency: Open source tools provide transparency in their work because their source code is freely available for anyone to inspect.

Community support: Open source projects often have vibrant communities of developers who contribute to their improvement and provide support to users.

Customisation: Users can modify open source tools according to their needs and preferences.

Cost-effectiveness: Open source tools are typically free to use, which can significantly lower the barrier to entry for individuals and organisations, especially those with limited budgets.

Security: While it might seem counterintuitive, open source tools often have better security than proprietary software.

And the cons are:

Quality variability: The quality of open source tools can vary widely depending on factors such as community support, maintenance efforts, and the expertise of contributors.

Support limitations: Users may need help obtaining timely assistance for complex issues or niche use cases.

Integration complexity: Integrating open source tools into existing workflows or ecosystems can sometimes be more complex than proprietary solutions. Compatibility issues, dependencies, and configuration requirements may pose challenges for users, especially those with limited technical expertise or resource constraints.

Dependency management: Open source tools often rely on external dependencies or libraries, introducing additional complexity and potential vulnerabilities into projects.

Popular open source code security tools

Here, I will take a few examples of open source code security tools for developers to use in their projects. Each tool has its strengths and focuses, so it's essential to choose the one that best fits the needs of your specific project and programming language.

OWASP ZAP (Zed Attack

Proxy): ZAP is a popular security tool for finding vulnerabilities in web applications during development and testing. It helps identify common security risks such as injection flaws, cross-site scripting (XSS), and broken authentication.

Brakeman: Brakeman is a static analysis tool for Ruby on Rails applications. It scans Rails applications for security vulnerabilities, including SQL injection, cross-site scripting (XSS), and mass assignment vulnerabilities.

FindBugs: FindBugs is a static analysis tool for Java code. It detects potential bugs and security vulnerabilities in Java applications, including issues like null pointer dereferences, resource leaks, and insecure random number generation.

SonarQube: SonarQube is an open source platform for continuously inspecting code quality and security. It supports multiple programming languages and provides a range of security-focused plugins for detecting vulnerabilities, code smells, and other issues.

Gosec: Gosec is a Go (Golang) code security scanner. It helps identify security issues in Go programs, such as hard coded credentials, insecure use of cryptographic functions, and potential vulnerabilities in third-party dependencies.

The best Python-focused code security tools

The tools listed below are just a few open source code security tools available for Python. Using them helps ensure that your Python code is secure and free from vulnerabilities.

It's important to remember that no single tool is perfect, and it's essential to use a combination of tools to comprehensively scan your code for security vulnerabilities. Static code analysis tools should also be used with other security practices, such as penetration testing and secure coding practices. Since listing every tool in this area is not possible, I would like to invite you to check <https://github.com/Rohith-sreedharan/OSFY-Code-Analyzer/> for a complete collection. You will get a comprehensive list of all the tools you can use.

▪ Bandit

This static application security testing (SAST) tool is designed specifically for Python applications. It helps Python developers identify and address security issues within their codebase by scanning code for common vulnerabilities like SQL injection, insecure direct object references, and more.

Link: <https://bandit.readthedocs.io/en/1.7.3/>

Installation: `pip3 install bandit`

Usage: `bandit -r <path/to/your/code>`

▪ Safety

This is another free and open source SAST tool that helps find vulnerabilities in Python code. Safety integrates with many popular build tools and CI/CD pipelines, making it easy to automate security scanning as part of your development process.

Link: <https://pypi.org/project/safety/>

Installation: `pip install safety`

Usage: `safety check`